Sesión 4
WORD EMBEDDINGS: WORD2VEC Y GLOVE

De vectores dispersos a representaciones densas y semánticas

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Word Embeddings: Word2Vec y GloVe **3**
1.1. Logro de la sesión . . . 3
1.2. Limitaciones de las representaciones dispersas . . . 3
1.2.1. Maldición de la dimensionalidad . . . 3
1.2.2. Falta de semántica . . . 3
1.2.3. Pérdida de relaciones . . . 4
1.2.4. Soluciones preliminares: LSA . . . 4
1.3. Word Embeddings: concepto fundamental . . . 4
1.3.1. Propiedades deseables . . . 4
1.3.2. La hipótesis distribucional . . . 4
1.4. Word2Vec . . . 4
1.4.1. CBOW (Continuous Bag of Words) . . . 4
1.4.2. Skip-gram . . . 5
1.4.3. Comparación CBOW vs Skip-gram . . . 6
1.5. GloVe (Global Vectors) . . . 6
1.5.1. Matriz de co-ocurrencia . . . 7
1.5.2. Fundamento matemático . . . 7
1.5.3. Entrenamiento . . . 7
1.5.4. Ventajas . . . 7
1.6. Propiedades de los embeddings . . . 7
1.6.1. Relaciones semánticas y sintácticas . . . 7
1.6.2. Evaluación de embeddings . . . 8
1.6.3. Vizualización de embeddings . . . 8
1.7. Embeddings contextuales vs. estáticos . . . 8
1.7.1. Embeddings estáticos . . . 8
1.7.2. Embeddings contextuales . . . 8
1.8. Caso integrado: Entrenamiento y visualización de embeddings con Gensim 9
1.8.1. Paso 1: Preparar el corpus . . . 9
1.8.2. Paso 2: Entrenar modelo Word2Vec . . . 9
1.8.3. Paso 3: Explorar embeddings . . . 9
1.8.4. Paso 4: Visualización con t-SNE . . . 9
1.8.5. Paso 5: Cargar embeddings pre-entrenados (GloVe) . . . 10
1.9. Preparación para entrevistas . . . 10
1.9.1. Preguntas típicas . . . 10
1.9.2. Preguntas avanzadas . . . 11
1.10. Conexión con el resto del curso . . . 11

1.11. Resumen 11

Capítulo 1

Word Embeddings: Word2Vec y GloVe

En la sesión anterior se trabajó con representaciones dispersas (BOW, TF-IDF) que, aunque útiles, presentan limitaciones fundamentales: alta dimensionalidad, falta de semántica y pobre generalización. Esta sesión introduce los **word embeddings**: representaciones densas y de baja dimensión que capturan relaciones semánticas y sintácticas entre palabras. Se exploran los dos modelos más influyentes: Word2Vec (con sus arquitecturas Skip-gram y CBOW) y GloVe, basado en factorización de matrices de co-ocurrencia. Se analizan sus fundamentos matemáticos, propiedades y aplicaciones.

1.1. Logro de la sesión

Comprender el concepto de embeddings densos y entrenar/visualizar word embeddings, interpretando sus propiedades semánticas y su relevancia en NLP moderno.

1.2. Limitaciones de las representaciones dispersas

1.2.1. Maldición de la dimensionalidad

Las representaciones tipo BOW generan vectores de dimensionalidad igual al tamaño del vocabulario ($|V|$), que puede ser fácilmente de 100,000 o más. Esto provoca:

- **Alta esparsidad**: la mayoría de las componentes son cero.
- **Problemas computacionales**: almacenamiento y procesamiento ineficientes.
- **Sobreajuste**: los modelos tienen demasiados parámetros para la cantidad de datos.

1.2.2. Falta de semántica

En BOW y TF-IDF, palabras como "gato" y "felino" son tratadas como dimensiones completamente independientes. El modelo no puede generalizar de una a otra. Si el corpus contiene "gato" pero no "felino", no podrá entender que "felino" es similar.

1.2.3. Pérdida de relaciones

No se capturan relaciones como sinonimia, hiperonimia o analogías. Por ejemplo, la relación "rey" es a "reina" como "hombre" es a "mujer" no es detectable.

1.2.4. Soluciones preliminares: LSA

Latent Semantic Analysis (LSA) aplica SVD a la matriz TF-IDF para obtener representaciones densas de menor dimensión. Sin embargo, tiene limitaciones: es puramente lineal y no captura bien las relaciones semánticas complejas.

1.3. Word Embeddings: concepto fundamental

Un **word embedding** es una representación de cada palabra como un vector denso de dimensión fija (típicamente 50-300) en un espacio continuo. La idea central es que palabras con significados similares deben tener representaciones cercanas en ese espacio.

1.3.1. Propiedades deseables

- **Dimensionalidad reducida**: típicamente 100-300, mucho menor que $|V|$.
- **Densidad**: todas las componentes son no nulas (generalmente).
- **Continuidad**: el espacio es continuo, permitiendo interpolaciones.
- **Estructura semántica**: las relaciones entre palabras se reflejan como operaciones vectoriales.

1.3.2. La hipótesis distribucional

La base teórica de los embeddings es la **hipótesis distribucional** de Harris (1954): "palabras que ocurren en los mismos contextos tienden a tener significados similares". Es decir, el significado de una palabra puede inferirse de las palabras que la rodean.

1.4. Word2Vec

Word2Vec, desarrollado por Mikolov et al. en 2013, es un modelo neuronal para aprender embeddings a partir de grandes corpus. Existen dos arquitecturas principales: CBOW (Continuous Bag of Words) y Skip-gram.

1.4.1. CBOW (Continuous Bag of Words)

Idea fundamental

CBOW predice la palabra objetivo a partir de su contexto (palabras circundantes). Por ejemplo, dado un contexto de tamaño 2 a cada lado: ["el", "gato", "al", "ratón"], se predice la palabra central "persigue".

Arquitectura

- **Entrada**: vectores one-hot de las palabras de contexto (cada una de tamaño $|V|$).
- **Capa oculta**: se promedian los embeddings de las palabras de contexto. La matriz de pesos entre entrada y oculta es la matriz de embeddings W (tamaño $|V| \times d$).
- **Capa de salida**: se calcula la similitud entre el vector promedio y cada palabra del vocabulario mediante otra matriz W' (tamaño $d \times |V|$). Se aplica softmax para obtener una distribución de probabilidad sobre el vocabulario.

Formalización matemática

Dado un contexto de C palabras $\{w_{c1}, w_{c2}, ..., w_{cC}\}$ alrededor de la palabra objetivo w_t:

1. Se obtienen los embeddings de cada palabra de contexto: $\mathbf{v}_{c_i} = W[:, w_{c_i}]$ (fila de W correspondiente a la palabra).
2. Se promedian: $\mathbf{h} = \frac{1}{C} \sum_{i=1}^{C} \mathbf{v}_{c_i}$.
3. Se calculan las puntuaciones para cada palabra: $\mathbf{u}_j = \mathbf{h}^T W'[:, j]$.
4. Se aplica softmax: $P(w_j|\text{contexto}) = \frac{\exp(\mathbf{u}_j)}{\sum_{k=1}^{|V|} \exp(\mathbf{u}_k)}$.
5. La pérdida es la entropía cruzada negativa entre la distribución predicha y la verdadera (one-hot de w_t):
$$\mathcal{L} = -\log P(w_t|\text{contexto})$$

Entrenamiento

Se utiliza backpropagation y gradiente descendente estocástico para actualizar tanto W como W'. Tras el entrenamiento, la matriz W contiene los embeddings de las palabras.

Ventajas y desventajas

- **Ventaja**: más rápido de entrenar que Skip-gram para el mismo corpus.
- **Desventaja**: no funciona tan bien con palabras poco frecuentes.

1.4.2. Skip-gram

Idea fundamental

Skip-gram invierte el problema: dada la palabra objetivo, predice las palabras de contexto. Por ejemplo, dada "persigue", predice "el", "gato", "al", "ratón".

Arquitectura

- **Entrada**: vector one-hot de la palabra objetivo.
- **Capa oculta**: se obtiene el embedding de la palabra objetivo: $\mathbf{h} = W[:, w_t]$.
- **Capa de salida**: para cada posición de contexto j, se calcula la probabilidad de que la palabra en esa posición sea w mediante softmax sobre el vocabulario.

Formalización matemática

Para una palabra objetivo w_t y una palabra de contexto w_c en una ventana:

$$P(w_c|w_t) = \frac{\exp(\mathbf{v}_{w_t}^T \mathbf{u}_{w_c})}{\sum_{k=1}^{|V|} \exp(\mathbf{v}_{w_t}^T \mathbf{u}_k)}$$

donde $\mathbf{v}_{w_t}$ es el embedding de entrada (fila de W) y $\mathbf{u}_{w_c}$ es el embedding de salida (columna de W').

La función de pérdida para un par (objetivo, contexto) es:

$$\mathcal{L} = -\log P(w_c|w_t)$$

y se suma sobre todas las posiciones de contexto.

Muestreo negativo (Negative Sampling)

Calcular el softmax sobre todo el vocabulario es prohibitivo. Skip-gram utiliza **muestreo negativo** para aproximar la pérdida. La idea es distinguir la palabra de contexto verdadera de unas pocas palabras negativas (ruido) muestreadas aleatoriamente.

La función de pérdida para un par (objetivo, contexto) con k muestras negativas es:

$$\mathcal{L} = -\log \sigma(\mathbf{v}_{w_t}^T \mathbf{u}_{w_c}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_{w_t}^T \mathbf{u}_{w_i})$$

donde σ es la función sigmoide y w_i son palabras muestreadas de una distribución de ruido (típicamente unigrama elevado a 3/4 para dar más peso a palabras frecuentes).

Ventajas y desventajas

- **Ventaja**: funciona mejor con palabras poco frecuentes que CBOW.
- **Desventaja**: más lento de entrenar.

1.4.3. Comparación CBOW vs Skip-gram

Característica	**CBOW**	**Skip-gram**
Dirección	Contexto → Objetivo	Objetivo → Contexto
Velocidad de entrenamiento	Más rápida	Más lenta
Rendimiento con palabras frecuentes	Bueno	Bueno
Rendimiento con palabras poco frecuentes	Regular	Excelente
Uso típico	Corpus grandes, eficiencia	Corpus pequeños, palabras r

Cuadro 1.1: Comparación entre CBOW y Skip-gram.

1.5. GloVe (Global Vectors)

GloVe, desarrollado por Pennington et al. en 2014, combina ideas de Word2Vec con factorización de matrices de co-ocurrencia.

1.5.1. Matriz de co-ocurrencia

Se construye una matriz X de tamaño $|V| \times |V|$ donde X_{ij} es el número de veces que la palabra j aparece en el contexto de la palabra i (en una ventana de tamaño fijo). A diferencia de Word2Vec, que solo usa ventanas locales, GloVe explota estadísticas globales del corpus.

1.5.2. Fundamento matemático

La hipótesis de GloVe es que los embeddings deben satisfacer:

$$\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j \approx \log X_{ij}$$

para palabras que co-ocurren frecuentemente, donde b_i y b_j son sesgos.

La función de pérdida es una suma ponderada de errores cuadráticos:

$$\mathcal{L} = \sum_{i,j=1}^{|V|} f(X_{ij})(\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j - \log X_{ij})^2$$

donde f es una función de ponderación que da menos peso a co-ocurrencias muy raras y muy frecuentes:

$$f(x) = \begin{cases} (x/x_{\text{máx}})^\alpha & \text{si } x < x_{\text{máx}} \\ 1 & \text{en otro caso} \end{cases}$$

con $x_{\text{máx}}$ típicamente 100 y $\alpha = 0.75$.

1.5.3. Entrenamiento

Se optimiza mediante gradiente descendente. A diferencia de Word2Vec, GloVe procesa todo el corpus de una vez para construir la matriz de co-ocurrencia, y luego entrena en los pares no nulos.

1.5.4. Ventajas

- Aprovecha estadísticas globales del corpus.
- Produce embeddings con buenas propiedades de analogía.
- Más rápido de entrenar que Skip-gram en algunos casos.

1.6. Propiedades de los embeddings

1.6.1. Relaciones semánticas y sintácticas

Una de las propiedades más sorprendentes de los embeddings es que las relaciones entre palabras se manifiestan como operaciones vectoriales. Por ejemplo:

$$\mathbf{v}_{\text{rey}} - \mathbf{v}_{\text{hombre}} + \mathbf{v}_{\text{mujer}} \approx \mathbf{v}_{\text{reina}}$$

Esto indica que el embedding de "reina" es cercano al resultado de la operación.

Otros ejemplos:

- Madrid - España + Francia París
- caminar - caminó + nadó nadar
- mejor - bueno + malo peor

1.6.2. Evaluación de embeddings

Las analogías se utilizan como tarea de evaluación intrínseca. Un conjunto de prueba típico contiene pares como:

$$(\text{"París"}, \text{"Francia"}) : (\text{"Roma"}, \text{"Italia"})$$

Se evalúa si el embedding de "Italia" es el más cercano a $\mathbf{v}_{\text{París}} - \mathbf{v}_{\text{Francia}} + \mathbf{v}_{\text{Roma}}$.

1.6.3. Vizualización de embeddings

Para visualizar embeddings (típicamente en 2D o 3D), se utilizan técnicas de reducción de dimensionalidad como t-SNE o PCA. Por ejemplo, proyectando embeddings de países y capitales, se observan agrupaciones por región.

1.7. Embeddings contextuales vs. estáticos

1.7.1. Embeddings estáticos

Word2Vec y GloVe producen un único embedding por palabra, independientemente del contexto. Esto tiene limitaciones:

- No resuelven la polisemia: la palabra "banco" tiene el mismo embedding en "banco de peces" que en "banco de inversión".
- No capturan significado dependiente del contexto.

1.7.2. Embeddings contextuales

Modelos posteriores (ELMo, BERT, GPT) generan embeddings que dependen del contexto. Cada aparición de una palabra puede tener un vector diferente, calculado en función de las palabras circundantes. Esto permite manejar la polisemia y capturar matices contextuales.

Evolución:

- Estáticos $\rightarrow$ Contextuales (ELMo, 2018)
- Contextuales con Transformers (BERT, 2018)
- Modelos generativos (GPT)

1.8. Caso integrado: Entrenamiento y visualización de embeddings con Gensim

1.8.1. Paso 1: Preparar el corpus

Usaremos el dataset de noticias de 20 Newsgroups (aproximadamente 20,000 documentos). Preprocesamos con tokenización, minúsculas y eliminación de puntuación.

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt')
import pandas as pd

# Cargar datos (simulado)
sentences = [
    "el gato persigue al rat n",
    "el perro persigue al gato",
    "el rat n huye del gato"
]
tokenized = [word_tokenize(s.lower()) for s in sentences]
```

1.8.2. Paso 2: Entrenar modelo Word2Vec

```
model = Word2Vec(
    sentences=tokenized,
    vector_size=100,      # dimensi n de los embeddings
    window=5,              # ventana de contexto
    min_count=1,           # ignorar palabras con frecuencia <
       min_count
    workers=4,             # paralelizaci n
    sg=1                   # 1 para Skip-gram, 0 para CBOW
)
```

1.8.3. Paso 3: Explorar embeddings

```
# Obtener embedding de una palabra
vector_gato = model.wv['gato']

# Palabras m s similares
model.wv.most_similar('gato', topn=5)

# Analog as: rey - hombre + mujer     reina
result = model.wv.most_similar(positive=['mujer', 'rey'],
   negative=['hombre'])
```

1.8.4. Paso 4: Visualización con t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

words = list(model.wv.index_to_key)[:100]  # primeras 100
   palabras
vectors = [model.wv[word] for word in words]

tsne = TSNE(n_components=2, random_state=42)
vectors_2d = tsne.fit_transform(vectors)

plt.figure(figsize=(12,8))
for i, word in enumerate(words):
    plt.scatter(vectors_2d[i,0], vectors_2d[i,1])
    plt.annotate(word, (vectors_2d[i,0], vectors_2d[i,1]))
plt.show()
```

1.8.5. Paso 5: Cargar embeddings pre-entrenados (GloVe)

```
from gensim.scripts.glove2word2vec import glove2word2vec
from gensim.models import KeyedVectors

# Convertir GloVe a formato Word2Vec
glove2word2vec('glove.6B.100d.txt', 'glove.6B.100d.word2vec.txt')

# Cargar embeddings
glove = KeyedVectors.load_word2vec_format('glove.6B.100d.word2vec
   .txt')
```

1.9. Preparación para entrevistas

1.9.1. Preguntas típicas

- **"Explica la diferencia entre Skip-gram y CBOW."** CBOW predice la palabra objetivo a partir del contexto; Skip-gram predice el contexto a partir de la palabra objetivo. Skip-gram funciona mejor con palabras poco frecuentes, CBOW es más rápido.

- **"¿Qué es el muestreo negativo y por qué se usa?"** Técnica para aproximar el softmax en Word2Vec. En lugar de actualizar todas las palabras, solo se actualiza la palabra objetivo positiva y unas pocas negativas (ruido). Reduce drásticamente el costo computacional.

- **"¿Cómo interpretas la operación vectorial rey - hombre + mujer reina?"** Los embeddings codifican relaciones semánticas como vectores. La diferencia rey - hombre captura la relación de "masculino a femenino" (o "real a no real"). Sumar mujer aplica esa relación, dando cerca de reina.

- **"¿Cuál es la diferencia entre embeddings estáticos y contextuales?"** Estáticos: una representación fija por palabra, independiente del contexto. Contextuales: la representación varía según el contexto, resolviendo polisemia.
- **"¿En qué se diferencia GloVe de Word2Vec?"** Word2Vec usa ventanas locales y entrena con predicción; GloVe usa estadísticas globales de co-ocurrencia y optimiza una factorización matricial. Ambos producen embeddings similares, pero GloVe puede converger más rápido.

1.9.2. Preguntas avanzadas

- **"¿Por qué el muestreo negativo usa una distribución de ruido con exponente 3/4?"** Esto da más peso a palabras frecuentes en el muestreo, haciendo que el modelo las distinga mejor sin sobrecargar con las muy frecuentes.
- **"¿Qué problemas tienen los embeddings con palabras polisémicas?"** Al ser estáticos, mezclan todos los significados en un solo vector, lo que puede ser subóptimo para tareas que requieren discriminación de sentidos.

1.10. Conexión con el resto del curso

- **Sesión 5**: Modelos secuenciales (RNN, LSTM) que utilizan embeddings como entrada.
- **Sesión 6**: Mecanismos de atención y Transformers, que combinan embeddings con contexto.
- **Sesión 7**: Modelos pre-entrenados (BERT, GPT) que generan embeddings contextuales.

1.11. Resumen

- Los **word embeddings** son representaciones densas que capturan semántica.
- **Word2Vec** tiene dos arquitecturas: CBOW (contexto→objetivo, rápido) y Skip-gram (objetivo→contexto, mejor para palabras raras).
- **Muestreo negativo** acelera el entrenamiento de Skip-gram.
- **GloVe** factoriza la matriz de co-ocurrencia global, combinando ventajas de ambos.
- Los embeddings permiten **analogías** mediante operaciones vectoriales.
- Son la base de modelos más avanzados, aunque han sido superados por **embeddings contextuales**.